NLP HW4

Question 1: Constituency parsing

I experimented with the Berkeley Neural Parser (`benepar` 0.2.0) using its English `benepar_en3` model.

(a)

The parser produced Penn Treebank-style trees. Words were first assigned part-of-speech tags, then grouped into constituents such as `NP`, `VP`, `PP`, `SBAR`, and `S`, with the whole tree placed under `TOP`. I found its treatment of questions especially interesting!

For "Which report did the senator say the committee had rejected?", it separated the fronted phrase as a `WHNP` and put the rest of the question in an `SQ`, including the embedded clause. However, the tree did not explicitly show the missing object position after "rejected" or connect that gap to "which report." Coordination was also represented structurally: in "The chef cooked the chicken and served it with rice," the two verb phrases were sisters joined by `CC and`.

(b)

The parser handled several difficult ordinary constructions well. It correctly found the two coordinated verb phrases in the chef sentence, and it recognized the embedded clause in the long-distance question. It even correctly analyzed the garden-path sentence "The complex houses married and single soldiers and their families," treating "complex" as a noun and "houses" as the main verb.

One error involved attachment ambiguity. For "The lawyer interviewed the witness with the camera," I intended "with the camera" to modify "the witness." The parser instead attached that `PP` inside the main `VP`, favoring the reading that the lawyer used the camera to conduct the interview. It also failed on "The horse raced past the barn fell": instead of treating "raced past the barn" as a reduced relative clause modifying "horse" and "fell" as the main verb, it made "raced" and "fell" separate verb phrases under the sentence's main `VP`.

(c)

I constructed the sentence "The retired police the stations." My intended analysis is `[NP The retired] [VP police [NP the stations]]`: retired people are staffing or monitoring the stations. The words are familiar and the sentence is

grammatical, but the parser did not build that subject and predicate. It tagged "retired" as a past participle and "police" as a noun, then produced an essentially flat sentence containing "the stations" as a separate noun phrase. The sentence therefore confused the parser in the intended way by exploiting the much more common phrase "the retired police."

Question 2: Other kinds of parsers

(a) Dependency grammar

I tried spaCy's English dependency parser on several of the same sentences. In contrast to the Berkeley parser's nested phrase-structure trees, spaCy made individual words the nodes of the tree. Each word was connected to a head word by a labeled relation such as `nsubj`, `dobj`, `aux`, `conj`, `prep`, or `pobj`. For example, in "The chef cooked the chicken and served it with rice," it made "cooked" the root, attached "chef" as its subject and "chicken" as its direct object, and attached "served" to "cooked" with a `conj` relation. This compactly represented that the two coordinated verbs share the same subject without building explicit `NP` and `VP` constituents.

The dependency parser handled that coordination well, but it had some of the same weaknesses as the constituency parser. In "The lawyer interviewed the witness with the camera," the preposition "with" depended on "interviewed," so it again selected the instrument reading rather than my intended reading in which the witness had the camera. For the long-distance question, it correctly made "committee" the subject of "rejected" and made "rejected" a clausal complement of "say," but it attached "report" to "say" with the generic label `dep` rather than showing that it is the missing object of "rejected." Finally, it failed on "The retired police the stations" as intended: it treated "police" as the root noun and "stations" as an appositive instead of recognizing "police" as the verb. Thus, dependency parsing makes head-dependent relations very easy to inspect, but choosing the correct interpretation is still very difficult!

(b) Spanish

I also ran spaCy's Spanish dependency parser on sentences containing pro-drop, clitic pronouns, postverbal subjects, relative clauses, and the unusual argument structure of *gustar*. The Spanish parses had the same general style as the English dependency parses, in that the words were nodes connected to head words by labeled arcs, but some labels did differ. For example, the Spanish model used `obj` where the English model used `dobj`, and it represented prepositions such as *a* and *con* with `case` relations rather than making them heads with `prep` relations. In "El libro que me recomendaste fue fascinante," it also made the

adjective *fascinante* the root and treated *fue* as its copula, rather than making the verb the root.

The results appeared fairly accurate but not uniformly so. In the pro-drop sentence "Ayer compré un libro interesante," the parser correctly made *compré* the root and *libro* its object even though the first-person subject *yo* was omitted. It also correctly recognized the postverbal subject in "Llegaron los estudiantes tarde" and the grammatical subject *los libros* in "A Juan le gustan los libros," rather than incorrectly treating Juan as the subject. This is good, because these constructions idffer noticeably from ordinary English word order!

There were also clear errors. In "El libro que me recomendaste fue fascinante," *que* stands for the book and is the object of *recomendaste*, but the parser labeled it as that verb's subject. In "Llegaron los estudiantes tarde," it correctly found the subject but attached *tarde* to *estudiantes* instead of to *llegaron*, even though "late" describes when they arrived. In "María se lo dio a Juan," it identified the clitics *se* and *lo* as indirect and direct objects, respectively, but it also labeled Juan as another object, making the doubled recipient construction less clear. Overall, the Spanish parser used the same basic dependency framework as the English one and handled several language-specific constructions, but its local attachment and grammatical-role errors made it seem somewhat less reliable on these examples.

Question 3: Probabilistic Earley parser

(a) Correctness

I represented an item's structural identity with its grammar rule, dot position, start position, and end position, where the end position is basically just the chart column containing the item. Each unique item has one stable chart entry that stores the total weight and backpointer for its best derivation found so far. A predicted item starts with the weight of its own rule, $-\log_2 p(\text{rule})$, and scanning a terminal advances the dot without changing the weight. Attaching a completed constituent advances its customer and adds the completed constituent's weight to the customer's current weight. Therefore, when a rule is complete, its weight is the sum of the weights of that rule and all rules in its completed child subtrees.

A scan backpointer stores the previous dotted item and the terminal that we then scanned, and an attach backpointer stores the previous customer and the completed child. Predicted items have no backpointer because they cover an empty prefix. After parsing, I selected the lowest-weight complete `ROOT` item spanning the

whole sentence and recursively followed these backpointers to reconstruct its tree.

Items in the same column are duplicates when their rule, dot position, and start position are all the same. If a new derivation is heavier than or equal to the stored one, it is discarded, and if it is lighter, the parser replaces the record's weight and backpointer. If the item is not already waiting on the agenda, I enqueue its stable record again so that its improved weight can propagate to items that were previously built from the heavier derivation. Thus, the chart retains only one copy of each structural item while still handling late improvements correctly!

(b) Efficiency

For a fixed grammar, there are only a constant number of dotted rules. An item is determined by one of those dotted rules and its start and end positions, so there are $O(n^2)$ possible items. Each item stores only a constant-size weight, backpointer, and bookkeeping record, while rules are shared by integer index. The chart therefore uses $O(n^2)$ space.

Each column has a hash map from an item's structural key to its stable vector index. This makes duplicate detection, insertion, and best-weight replacement $O(1)$ on average. The agenda is a FIFO `VecDeque` of those indices, so enqueue and dequeue are also $O(1)$. Constant-time duplicate checks are necessary because many different derivations can propose the same item; a linear search through a column on every proposal would introduce another factor of n.

Under the question's assumption that no item needs reprocessing, every unique item is removed from its agenda once. Prediction and scanning take $O(1)$ time for a fixed grammar. An attachment performs the unspeeded linear search through the completed constituent's start column, which contains $O(n)$ items. There are $O(n^2)$ items that can trigger such work, giving $O(n^3)$ total time.

My implementation does handle reprocessing for correctness: a strict weight improvement updates the existing record and schedules it again. Consequently, an item may be processed more than once, so without the question's simplifying assumption this baseline implementation does not claim a worst-case $O(n^3)$ bound. The extra work is directly proportional to the number of successful weight improvements and the number of repeated scan, predict, or attachment operations.

Question 4: Faster parsing

The original parser keeps a separate dotted rule for every grammar rule. This means that if many rules begin in the same way, the parser repeatedly stores and processes almost identical partial rules. My biggest change was to combine the rules for each nonterminal into a weighted finite-state automaton. A useful way to picture this is as a network of paths: rules that begin the same way share the same path until they differ, and identical remaining portions are shared as well. Reaching the end of a path adds that rule's weight.

As a result, a chart item now points to one state in this network instead of one specific dotted rule. Predicting a category such as `NP` starts its one shared automaton rather than inserting every `NP` rule separately. This also handles the batch duplicate-prediction trick automatically, since there is only one initial state to insert and check for duplicates.

I then added the remaining compatible improvements one at a time. For scanning, each state keeps separate indexes for terminal and nonterminal transitions. The parser can look up the current word directly instead of checking every word that could leave that state. I also gave each chart column an index of which items are waiting for each nonterminal. When an `NP` is completed, for example, the parser can immediately find the items waiting for an `NP` instead of searching the whole column.

Different internal analyses can sometimes produce the same category over the same span of words. I merge these completed constituents and only process one again if a lower-weight derivation is found. Finally, I added left-corner filtering. Before predicting rules at a position, it checks which categories could actually begin with the next word and avoids paths that cannot possibly match it.

I compiled each version with `rustc -O`. I timed 50 complete runs over the first two sentences of `wallstreet.sen`; these measurements are cumulative, so every row includes the improvements in the rows above it. The times include program startup and reading the grammar.

implementation	time for 50 runs	marginal factor	factor over FSA
weighted FSA	8.01 s	--	1.00x
FSA + terminal specialization	7.53 s	1.06x	1.06x
previous + customer index	1.38 s	5.46x	5.80x
previous + completed-constituent merging	1.04 s	1.33x	7.70x

implementation	time for 50 runs	marginal factor	factor over FSA
previous + left-corner filtering	1.11 s	0.94x	7.22x

I also directly compared the original `parse.rs` with the first FSA version over 10 runs of the same input. The original parser took 8.57 seconds, while the FSA version took 1.56 seconds, so this first change alone was about 5.49x faster. The customer index produced the largest additional improvement because attachment had previously searched entire chart columns.

Left-corner filtering was correct, but it did not help this short benchmark. Its setup and hash lookups made the last version about 6% slower than the version immediately before it. The FSA had already removed most of the unnecessary prediction that left-corner filtering normally prevents. I kept this result in the table for the sake of scientific integrity! On the complete set of nine Wall Street sentences, the final parser took 0.94 seconds. FSA alone took 28.16 seconds on that same full input, so the combined improvements made the complete workload about 30 times faster.

The parser found an analysis for all nine sentences, but several analyses have clear problems. In sentence 5, *with financing deals* attaches high to the matrix verb phrase headed by *had*, although it more naturally describes how Ford would *follow suit*. In sentence 6, *that* is analyzed as a `WHNP` and *pay* as a verb, rather than treating *that* as a complementizer and *pay* as the subject noun of *was flat*. In sentence 7, the initial gerund clause *running the combined companies* should act as the subject of *is likely*, but the parse instead makes *running* the outer predicate and embeds the rest of the sentence beneath it. In sentence 8, *then decided* is incorrectly placed inside an `NP` as adjectival material modifying *the information*, rather than being coordinated with *said*. In sentence 9, *otherwise we* is grouped as one adverb phrase even though *we* is the subject of the following independent clause. These errors reflect lexical-category and attachment ambiguities in the grammar rather than search errors in the optimized parser.